2. Processes

Computer and Virtual Technologies Engineering
Software Engineering

Operating Systems

2025-2026

Universidad LOYOLA

Contents

1) Process Control Blocks (PCB)
2) Threads
3) Process states
4) Process control
   - Modes of execution
   - Process switching
5) Interprocess communication

Contents

1) Process Control Blocks (PCB)
2) Threads
3) Process states
4) Process control
   - Modes of execution
   - Process switching
5) Interprocess communication

Process Control Blocks

Several **definitions** of the term **process**:

- A program in execution.
- An instance of a program running on a computer.
- The entity that can be assigned to and executed on a processor.
- A unit of activity characterized by the execution of a sequence of instructions, a current state and an associated set of system resources.

We can see a process as an entity that consists of a group of elements. Two essential elements:

- **Program code.** It may be shared with other processes that are executing the same program.
- **Set of data.** It is associated with the code.

Process Control Blocks

At any given time while the program is executing, the process can be uniquely characterized by:

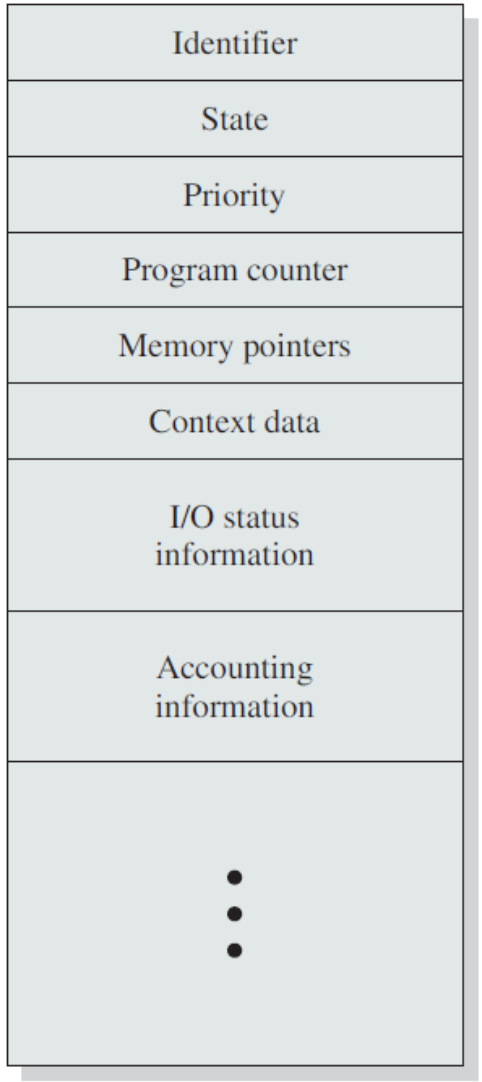
- **Identifier:** identifier of the process.
- **State**: current state.
- **Priority:** priority level relative to other processes.
- **Program counter:** address of the next instruction.
- **Memory pointers:** pointers to the program code, data and memory blocks shared with other processes.
- **Context data:** data in the registers of the processor.
- **I/O status information:** I/O requests, I/O devices assigned.
- **Accounting information:** amount of processor time and clock time used, time limits, account limits…

This information is stored in a data structure, called **Process Control Block (PCB).**

Process Control Blocks

Process Control Block (PCB)

- Created and managed by the OS.
- **Key tool** enabling **OS** to support **multiple processes** and multiprocessing.
- When a process is interrupted, the current values of the program counter and the processor registers (context data) are saved in the appropriate fields of the corresponding PCB, and the state of the process is changed to some other value (blocked or ready).



Contents

1) Process Control Blocks (PCB)
2) Threads
3) Process states
4) Process control
   - Modes of execution
   - Process switching
5) Interprocess communication

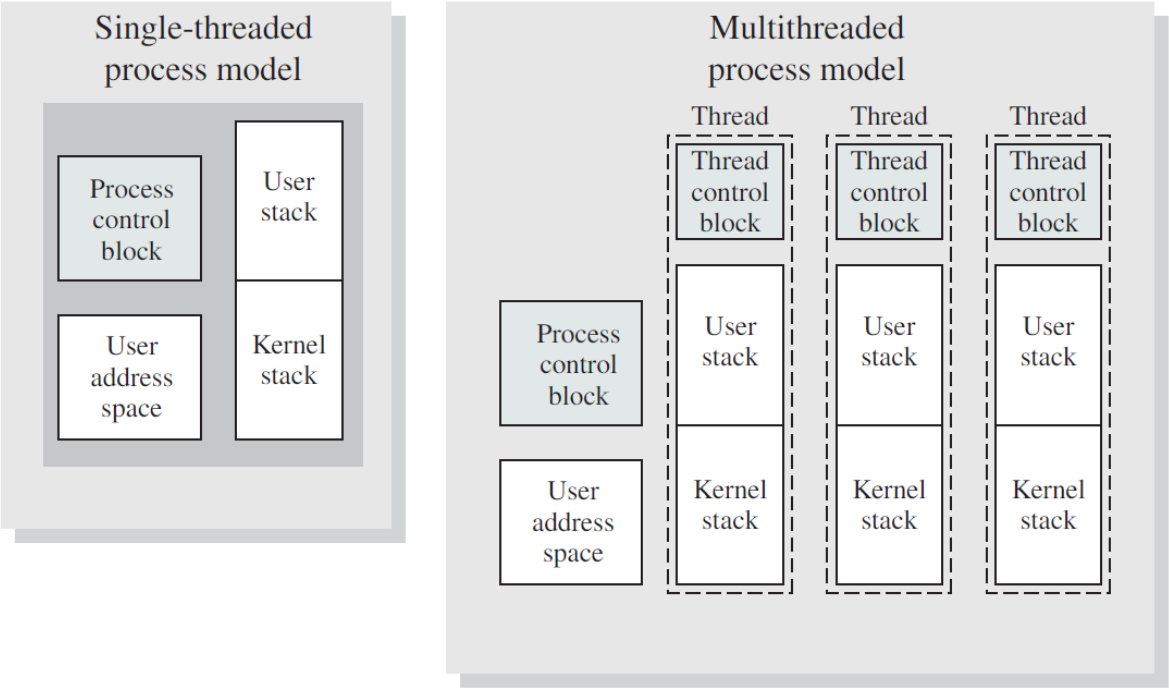
Threads

The concept of process has two characteristics:

- **Resource ownership:** from time to time, a process may be allocated control or ownership of resources (main memory, I/O channels, files…). The OS performs a protection function to prevent unwanted interferences between processes with respect to resources.
- **Scheduling/execution:** The execution of a process follows an execution path through one or more programs (interleaved with other processes). A process has a state and a dispatching priority.

These characteristics could be treated independently:

- Unit of **dispatching** is usually referred as a **thread**.
- Unit of **resource ownership** is usually referred as a **process**.



All the threads of a process share the state and resources of that process

Threads

The key benefits of **threads** derive from the **performance** implications:

- It takes far **less time** to **create** a new thread in an existing process, than to create a brand-new process.
- It takes **less time** to **terminate** a thread than a process.
- It takes **less time** to **switch** between two threads within the same process than to switch between processes.
- In most OS, communication between independent processes requires the intervention of the kernel to provide protection and the mechanisms needed for communication. However, threads within the same process share memory and files, they can communicate with each other without invoking the kernel. **Threads enhance efficiency in communication**.

Contents

1) Process Control Blocks (PCB)
2) Threads
3) Process states
4) Process control
   - Modes of execution
   - Process switching
5) Interprocess communication

Process states

We can characterize the behavior of an individual process by listing the sequence of instructions that execute. Such a listing is referred to as a **trace** of the process.

There is a small **dispatcher** program (switch the processor from one process to another).

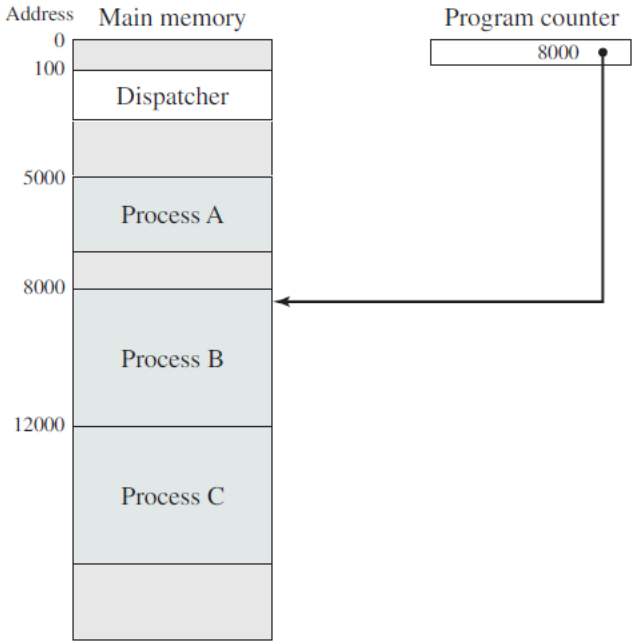
Example: First 6 instructions of process A are executed.

Time-out.

Execution of some code in the dispatcher (6 instructions before turning control to the process B).

After 4 instructions, process B requests I/O action.

The processor stops executing process B, and moves to C via the dispatcher…



5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011
(a) Trace of process A	(b) Trace of process B	(c) Trace of process C

5000 = Starting address of program of process A
8000 = Starting address of program of process B
12000 = Starting address of program of process C

Processor's point of view

1	5000		27	12004
2	5001		28	12005
3	5002		----------Time-out	
4	5003		29	100
5	5004		30	101
6	5005		31	102
----------Time-out			32	103
7	100		33	104
8	101		34	105
9	102		35	5006
10	103		36	5007
11	104		37	5008
12	105		38	5009
13	8000		39	5010
14	8001		40	5011
15	8002		----------Time-out	
16	8003		41	100
----------I/O request			42	101
17	100		43	102
18	101		44	103
19	102		45	104
20	103		46	105
21	104		47	12006
22	105		48	12007
23	12000		49	12008
24	12001		50	12009
25	12002		51	12010
26	12003		52	12011
			----------Time-out	

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

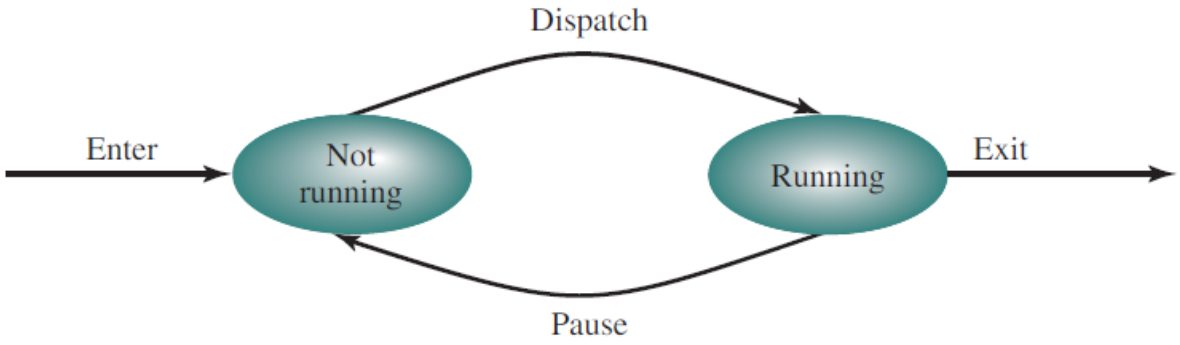
Process states

The operating system's principal responsibility is controlling the execution of processes, including:

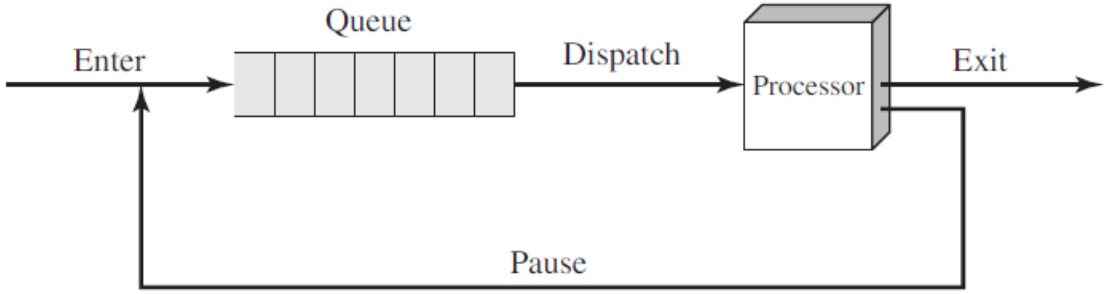
- Determining the interleaving pattern for execution.
- Allocating resources to processes.

First step for **designing** an **OS** is to describe the **behavior** that we would like the **processes** to exhibit.

Simplest possible model: **Two-State Process Model.**



(a) State transition diagram



(b) Queueing diagram

Process states

Process Creation

Reasons for process creation:

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Process states

Process Creation

OS builds the data structures used to manage the process and allocates address space in main memory.

When the OS creates a process at the request of another process, the action is referred as **process spawning** (**Parent** and **child** processes).

The related processes need to communicate and cooperate with each other.

Process spawning	*Parent process*	*Child process*
• When the OS creates a process at the explicit request of another process	• Is the original process	• Is the new process

Process states

Process Termination

Reasons for process termination:

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Process states

If all processes were always ready to execute, then the queueing suggested would be effective.

- Each process in the queue is given a certain amount of time, in turn, to execute and then returned to the queue, unless blocked.

This implementation is **inadequate**:

- Some process in the Not Running state are ready to execute, while **others** are **blocked** (i.e. waiting for an I/O operation to complete).
- How to handle this situation? Split the **Not Running** state in two states: **Ready** and **Blocked**.

Process states

Five-State Process Model

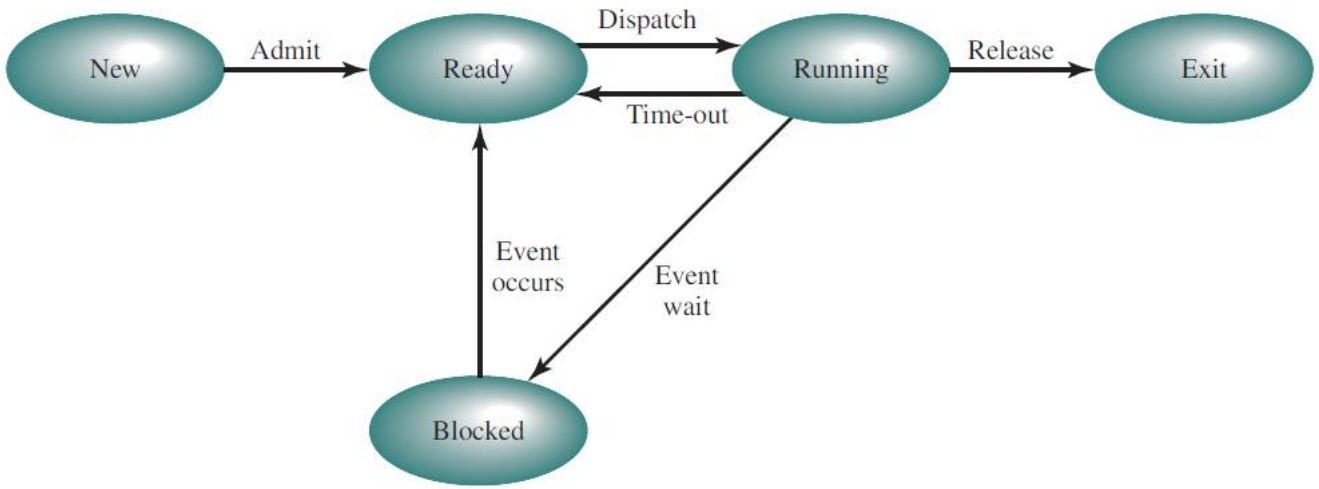
1. Running: the process that is currently being executed.

2. Ready: a process that is prepared to execute when given the opportunity.

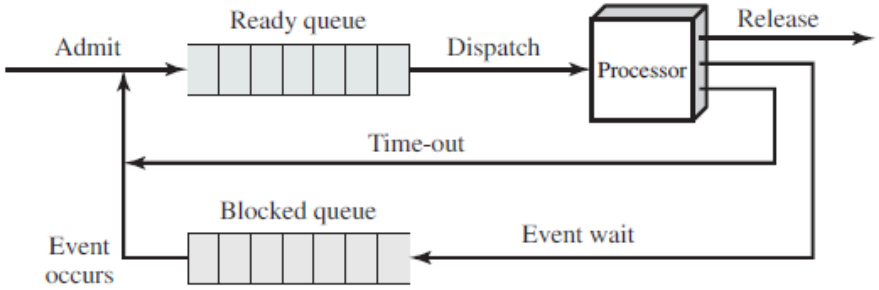
3. Blocked/Waiting: a process that cannot execute until some event occurs.

4. New: a process that has just been created but has not yet been admitted to the pool of executable processes by the OS.

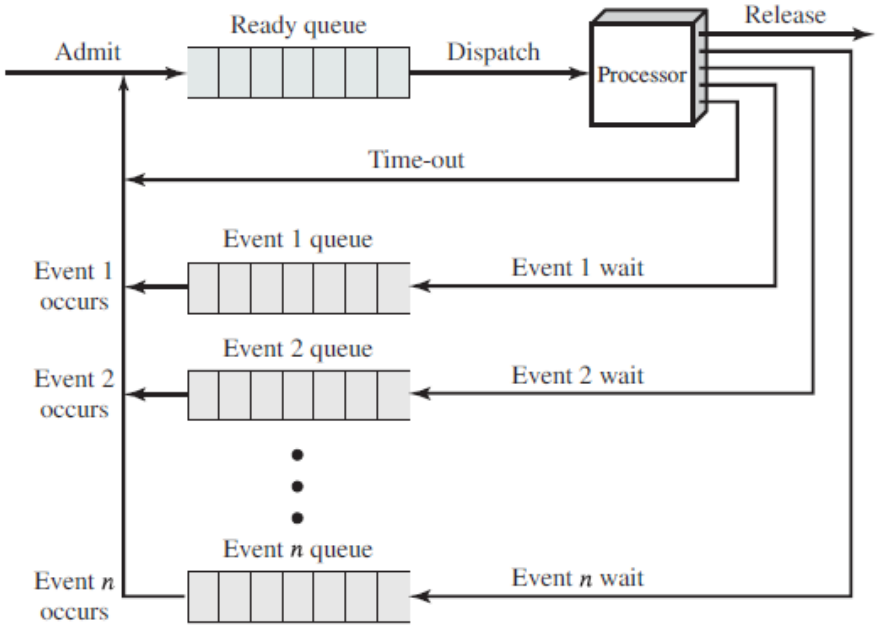
5. Exit: A process that has been released from the pool of executable processes by the OS.



Process states

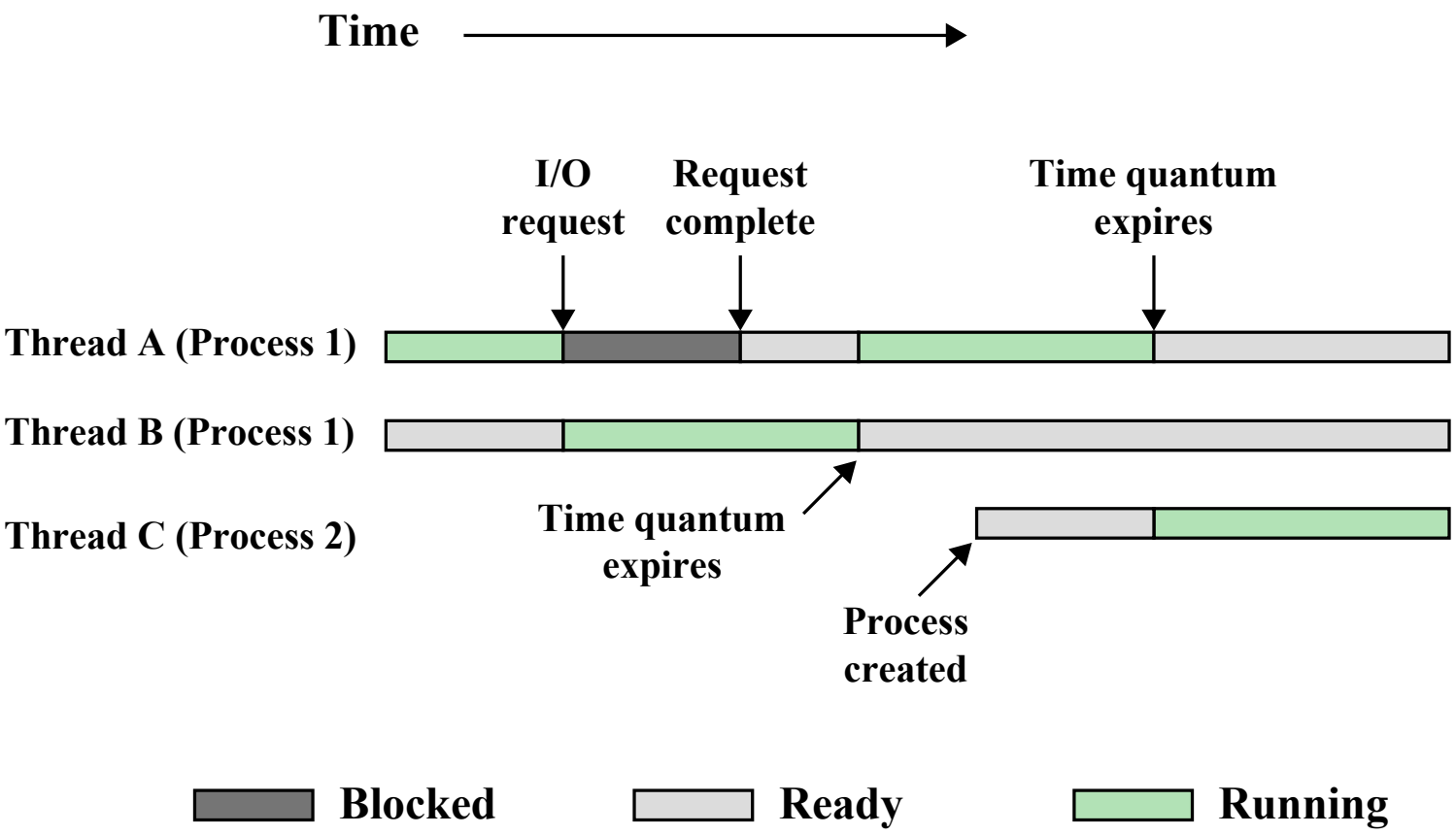


(a) Single blocked queue



(b) Multiple blocked queues

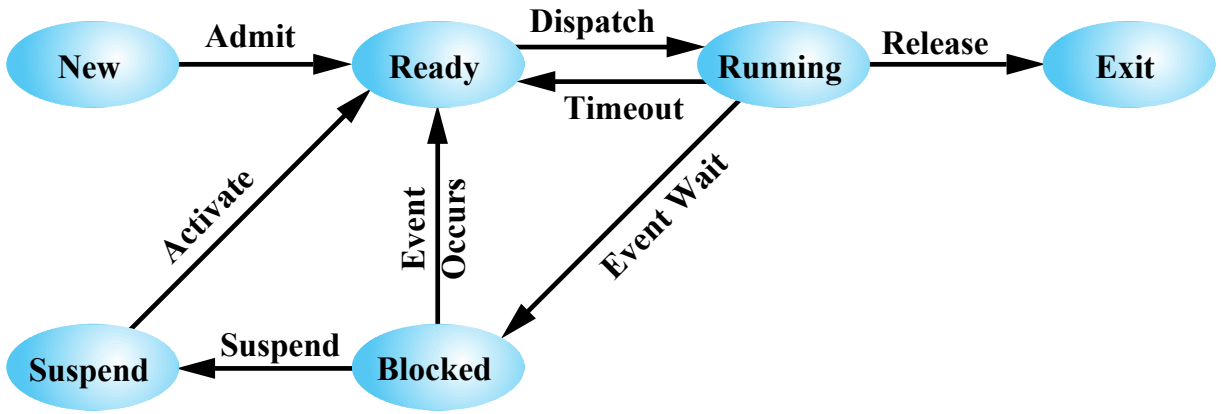
Process states



Process states

The suspend state

- Swapping: Involves moving part of all of a process from main memory to disk.
- When none of the processes in main memory is in the ready state, the OS swaps one of the blocked processes out on to disk into a suspend queue.
  - This is a queue of existing processes that have been temporarily kicked out of main memory or suspended.
  - The OS then brings in another process from the suspend queue or it honors a new-process request.
  - Execution then continues with the newly arrived process.



(a) With One Suspend State

Process states

The suspend state

Reasons for process suspension:

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

Contents

1) Process Control Blocks (PCB)
2) Threads
3) Process states
4) Process control
   - Modes of execution
   - Process switching
5) Interprocess communication

Process control: modes of execution

- Two modes:
  - **User mode**. Less privileged mode.
  - **Kernel mode (control or system mode)**. More privileged mode.
- Reason for two modes:
  - It is necessary to protect the OS and key operating system tables, such as PCB from interference by user programs.
  - In the kernel mode, the software has complete control of the processor and all its instructions, registers and memory.

Process control: modes of execution

Typical functions of an OS kernel (kernel mode):

Process management

- Process creation and termination.
- Process scheduling and dispatching.
- Process switching.
- Process synchronization and support for interprocess communication.
- Management of process control blocks.

Memory management

- Allocation of address space to processes.
- Swapping.
- Page and segment management.

I/O management

- Buffer management.
- Allocation of I/O channels and devices to processes.

Support functions

- Interrupt handling.
- Accounting.
- Monitoring.

Process control: process switching

When to switch processes?

It may occur any time that the OS has gained control from the currently running process.

Possible events that may give control to the OS:

Interrupt

- Cause: external to the execution of the current instruction.
- Use: reaction to an asynchronous external event.
- Examples:
  - Clock interrupt.
  - I/O interrupt.
  - Memory fault.

Trap

- Cause: associated with the execution of the current instruction.
- Use: handling of an error or an exception condition.

Supervisor call

- Cause: explicit request.
- Use: call to an operating system function.

Contents

1) Process Control Blocks (PCB)
2) Threads
3) Process states
4) Process control
   - Modes of execution
   - Process switching
5) Interprocess communication

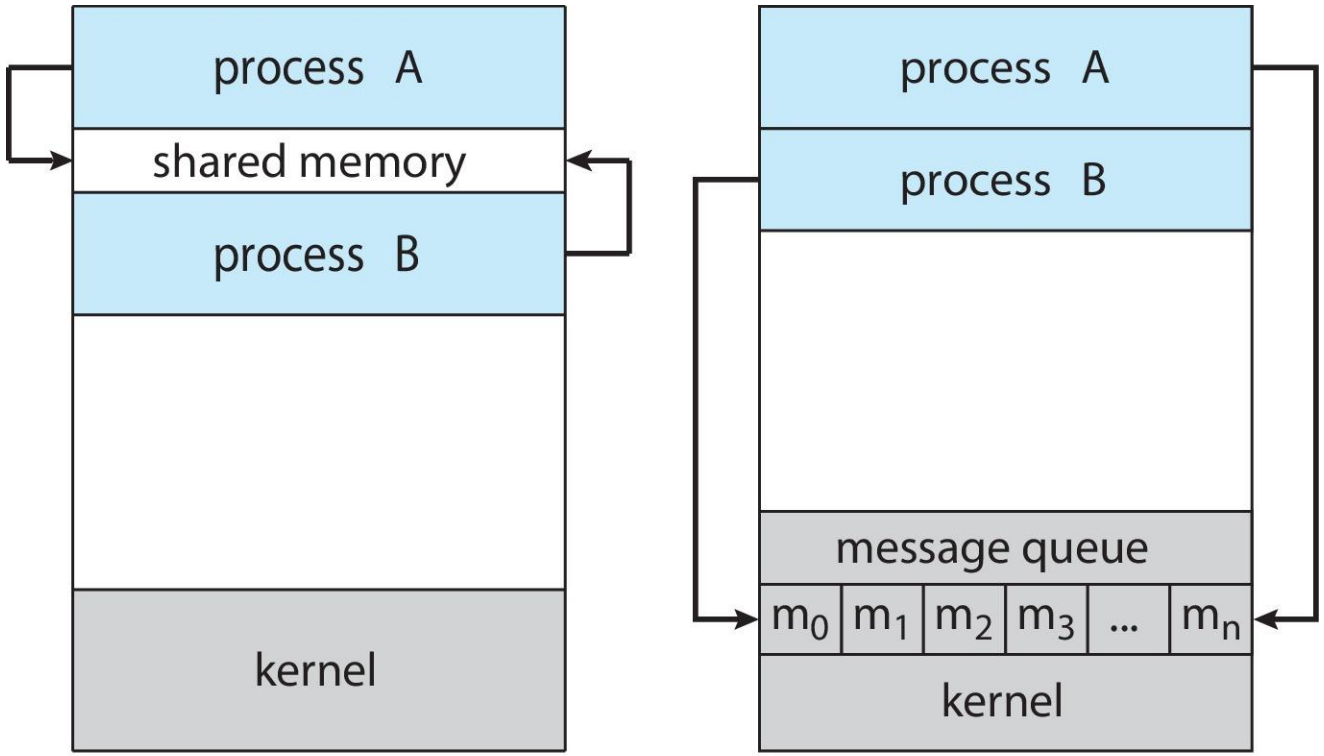
Interprocess communication

- Processes within a system may be **independent** or **cooperating.**
- Cooperating process can affect or be affected by other processes, including sharing data.
- Reasons for cooperating processes:
  - Information sharing.
  - Computation speedup.
  - Modularity.
  - Convenience.
- Cooperating processes need interprocess communication (IPC).
- Two models of IPC.
  - Shared memory.
  - Message passing.

Communications models

(a) Shared memory.

(b) Message passing.



Universidad
LOYOLA

Universidad
LOYOLA